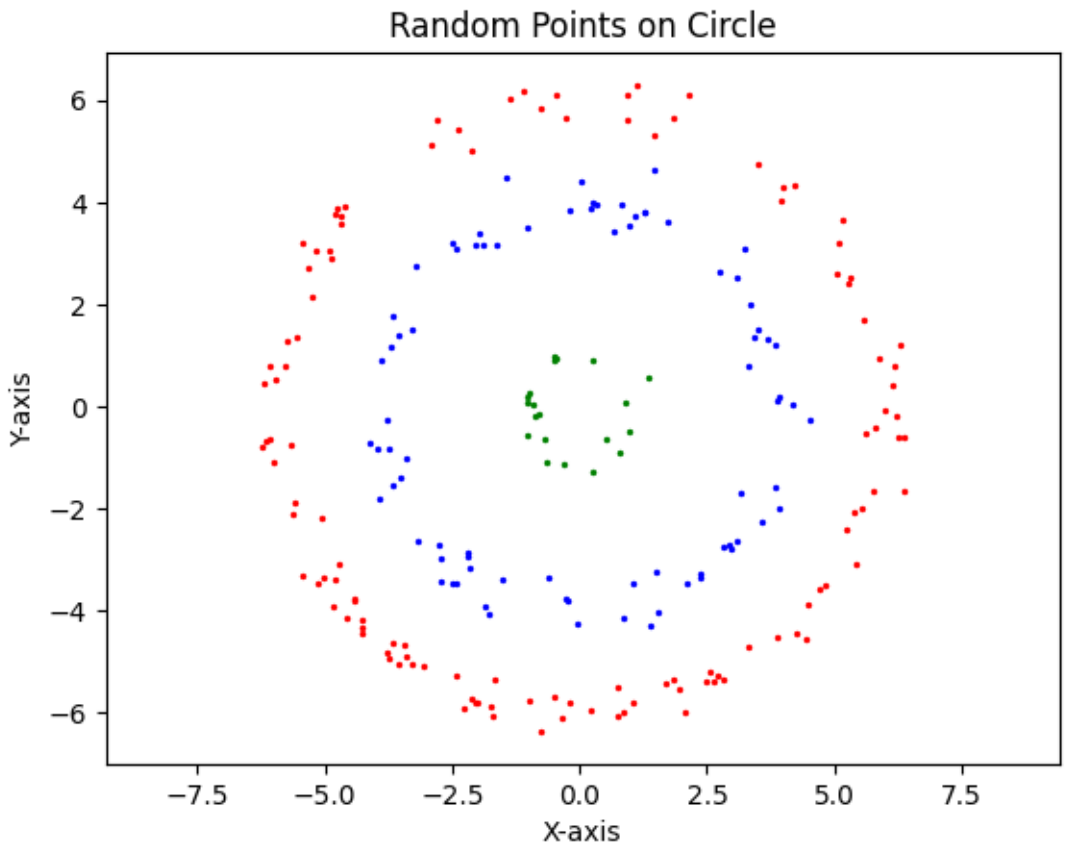
STAT672_Homework1_code

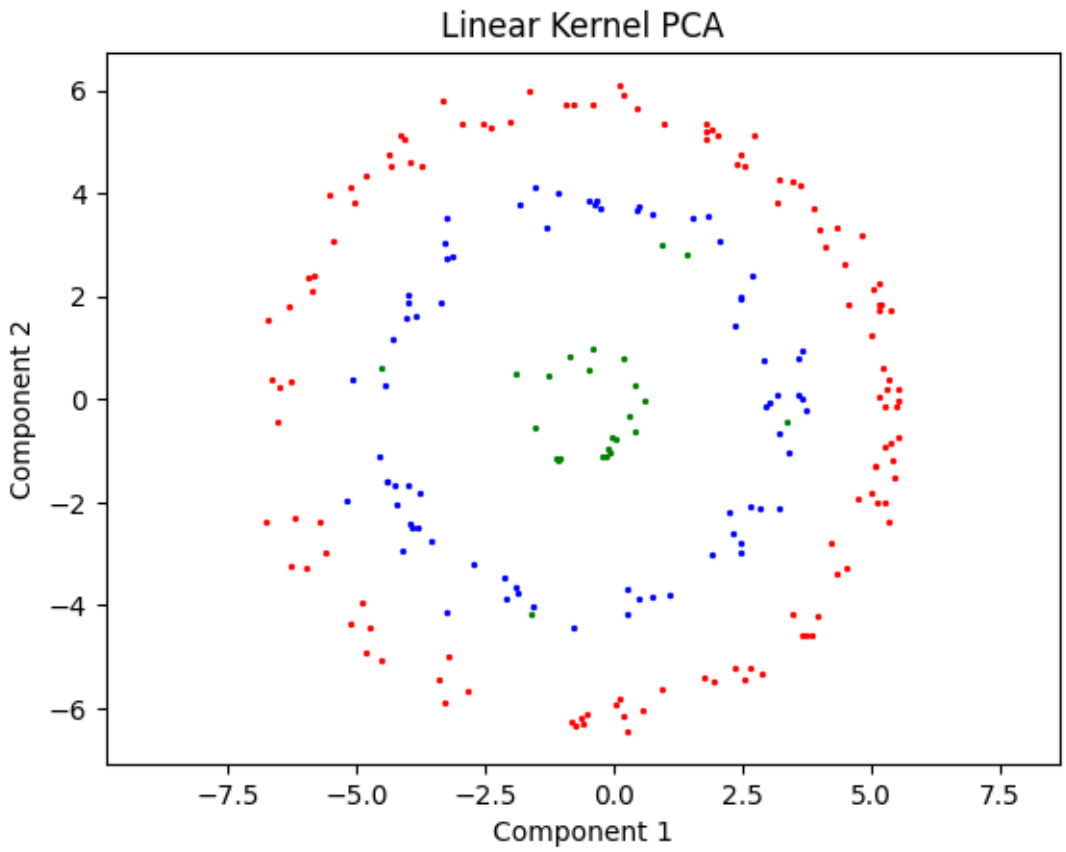
January 25, 2025

[17]:
```
import math
import random
import numpy as np
import matplotlib.pyplot as plt

#generate some random data
def generate_points_on_circle(radius, num_points,noise):
    xpoints=[]
    ypoints=[]
    for _ in range(num_points):
        # Generate a random angle in radians
        theta = random.uniform(0, 2 * math.pi)

        # Calculate x and y coordinates using polar coordinates
        x =  (radius+np.random.normal(0,noise,1)) * math.cos(theta)
        y =  (radius+np.random.normal(0,noise,1)) * math.sin(theta)

        xpoints.append(x)
        ypoints.append(y)

    return np.array(xpoints), np.array(ypoints)
#Parameteers
categories=3
factor=25
radius=[1,4,6]
color=['green','blue','red']
noise=.3
#Contruct X(:,0) annd X(:,1)
Xp=np.empty((0,1))
Yp=np.empty((0,1))

for c in range(categories):
    num_points=20*radius[c]
    xpoints,ypoints = generate_points_on_circle(radius[c],␣
 ↪int(num_points),noise)
    Xp=np.vstack((Xp,xpoints))
    Yp=np.vstack((Yp,ypoints))
```

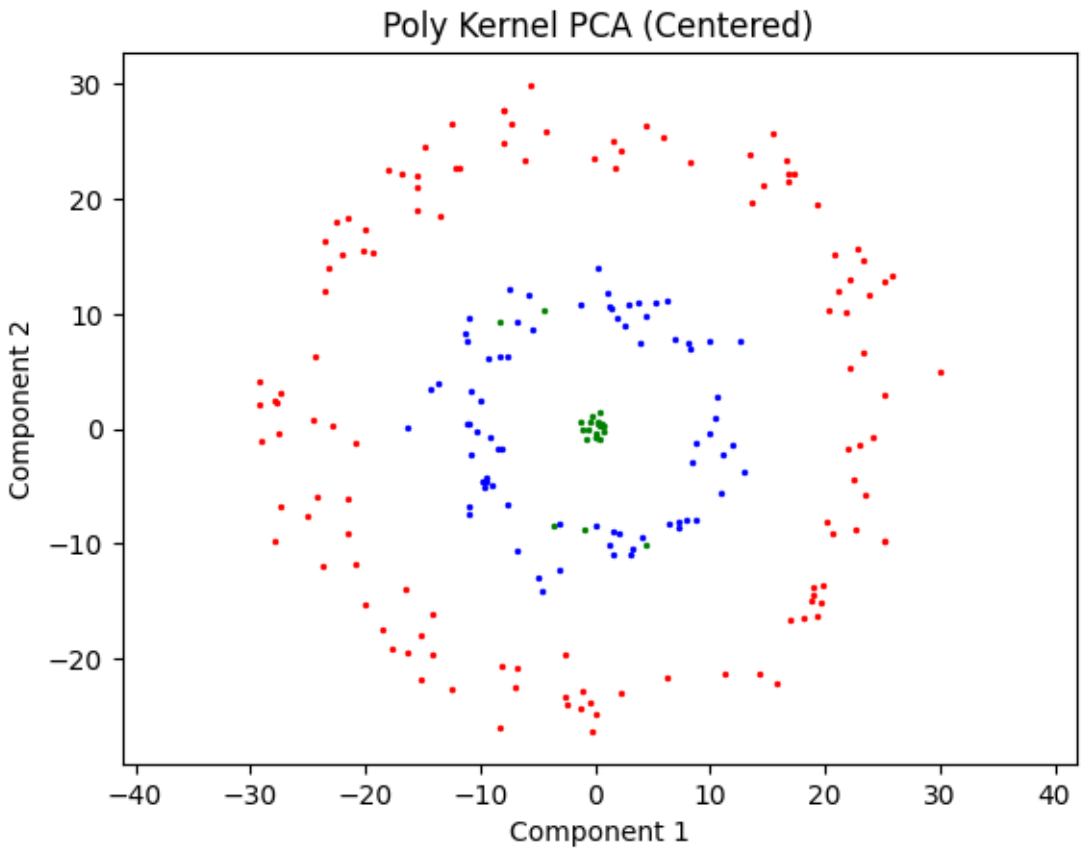
```
    #Scatter plot each color category
    plt.scatter(xpoints,ypoints,s=2,color=color[c])
plt.axis('equal')  # To maintain aspect ratio
plt.title('Random Points on Circle')
plt.xlabel('X-axis')
plt.ylabel('Y-axis')
plt.show()

#Construct X Matrix from data
X=np.hstack((Xp,Yp))
n_samples = X.shape[0]
One_matrix=np.ones((n_samples,n_samples))
Id_matrix=np.identity((n_samples))
#Variety of Kernels for PCA
def polynomial_kernel(X, Y, degree=2, c=1):
    return (np.dot(X, Y) + c) ** degree

def rbf_kernel(X, Y, gamma=1):
    return np.exp(-gamma * np.linalg.norm(X-Y)**2)

#Create Kernel Matrix
def create_kernel_matrix(X, kernel_func, **kwargs):
    n_samples = X.shape[0]
    One_matrix=np.ones((n_samples,n_samples))
    Id_matrix=np.identity((n_samples))
    K = np.zeros((n_samples, n_samples))
    for i in range(n_samples):
        for j in range(n_samples):
            K[i, j] = kernel_func(X[i], X[j], **kwargs)
    Kc=(Id_matrix-One_matrix/n_samples)@K@(Id_matrix-One_matrix/n_samples)
    return K,Kc

#Sort eigenvalues and eigenvectors from largest to smallest.
def eig_sorted(A):
    eigenvalues, eigenvectors = np.linalg.eig(A)
    idx = np.argsort(eigenvalues)[::-1]  # Sort in descending order
    eigenvalues = eigenvalues[idx]
    eigenvectors = eigenvectors[:, idx]
    return eigenvalues, eigenvectors
```



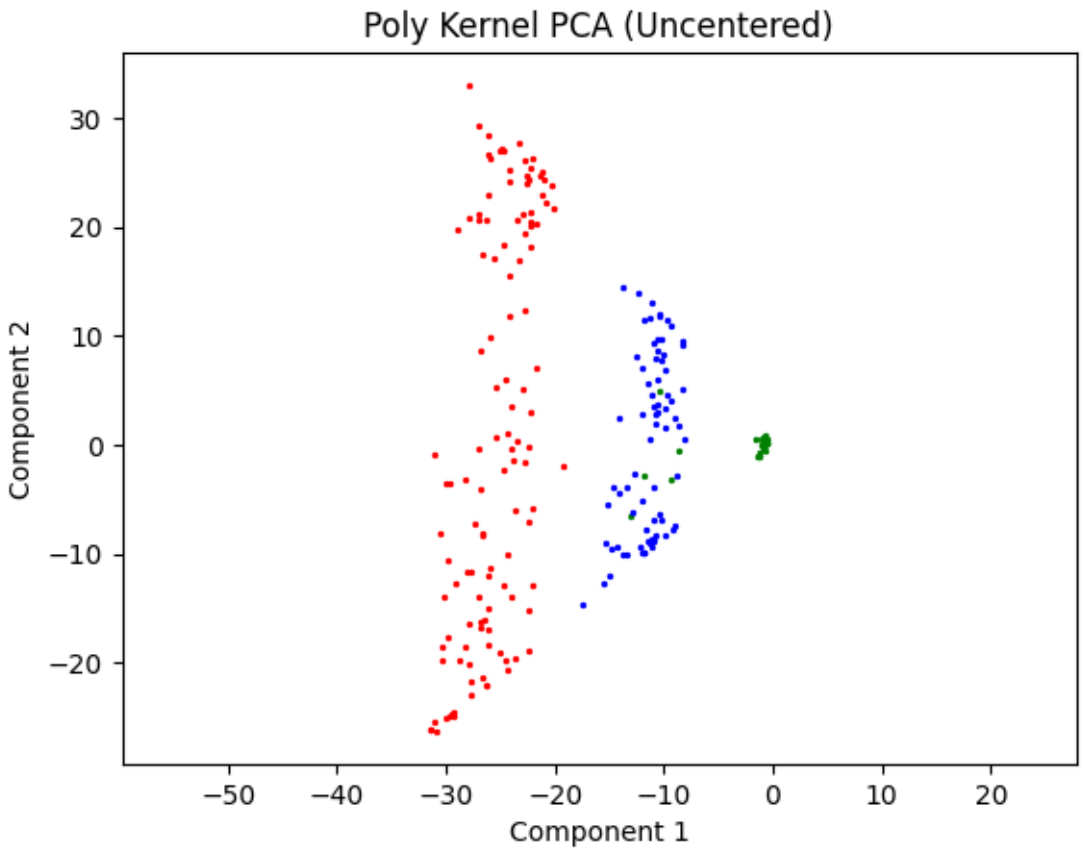
[26]:
```
#Linear PCA class method
K_poly,Kc_poly = create_kernel_matrix(X, polynomial_kernel, degree=1, c=0)
eigenvalues, eigenvectors=eig_sorted(Kc_poly)
pvalues=np.real(eigenvalues[0:2])
pvectors=np.real(eigenvectors[:,0:2])
D=np.diag(1/np.sqrt(pvalues))
PCA=Kc_poly@pvectors@D
inter=[slice(0,int(radius[0]*factor)),slice(int(radius[0]*factor),radius[1]*\
    factor),slice(radius[1]*factor,PCA.shape[0])]
for c in range(categories):
    plt.scatter(PCA[inter[c],0],PCA[inter[c],1],s=2,color=color[c])
plt.title("Linear Kernel PCA")
plt.xlabel('Component 1')
plt.axis('equal')
plt.ylabel('Component 2')
plt.show()
```



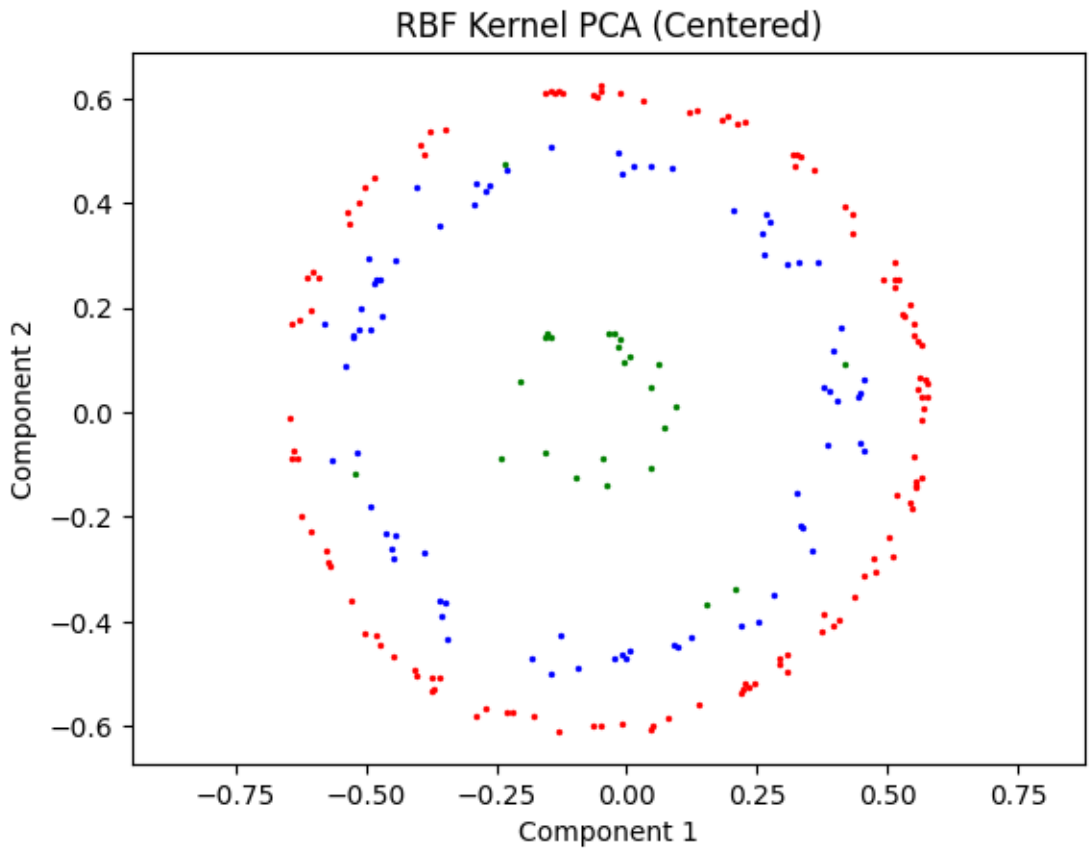
[27]:
```
# Polynomial PCA class method
K_poly,Kc_poly = create_kernel_matrix(X, polynomial_kernel, degree=2, c=1)
eigenvalues, eigenvectors=eig_sorted(Kc_poly)
pvalues=np.real(eigenvalues[0:2])
pvectors=np.real(eigenvectors[:,0:2])
D=np.diag(1/np.sqrt(pvalues))
PCA=K_poly@pvectors@D
inter=[slice(0,int(radius[0]*factor)),slice(int(radius[0]*factor),radius[1]*\
    factor),slice(radius[1]*factor,PCA.shape[0])]
for c in range(categories):
    plt.scatter(PCA[inter[c],0],PCA[inter[c],1],s=2,color=color[c])
plt.title("Poly Kernel PCA (Centered)")
plt.xlabel('Component 1')
plt.ylabel('Component 2')
plt.axis('equal')
plt.show()
```



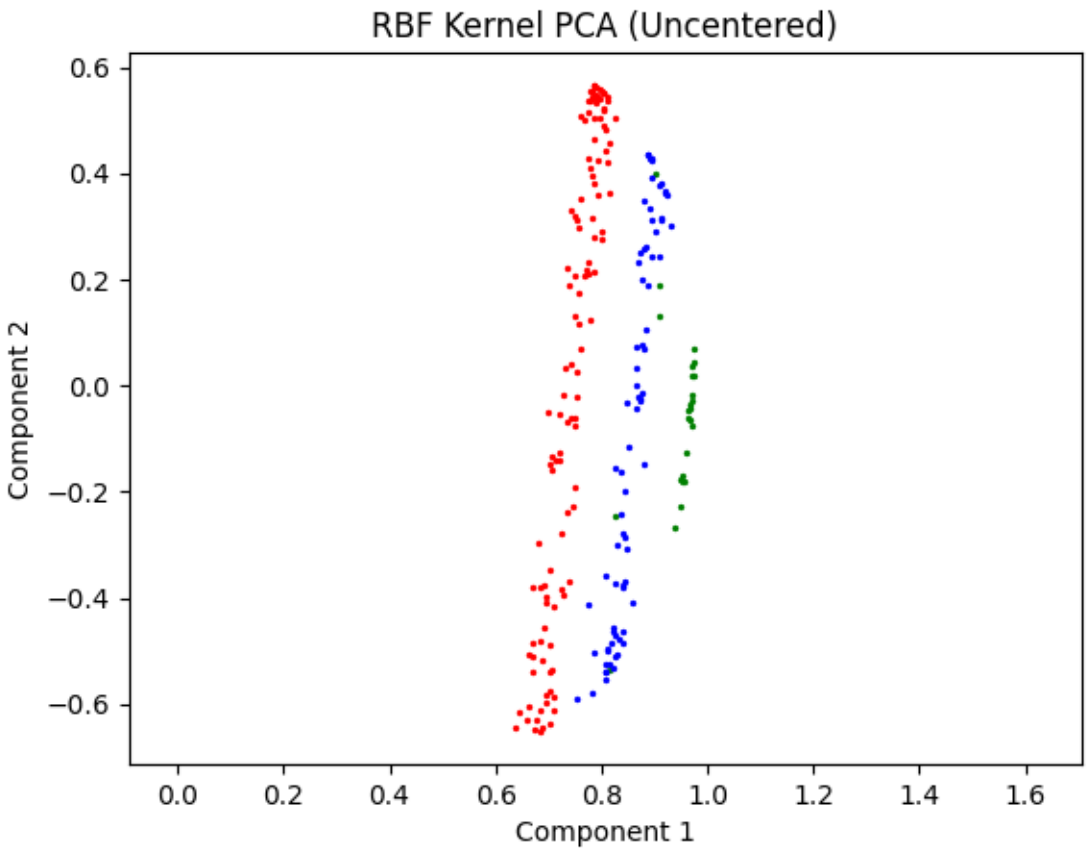
[28]:
```
# Polynomial PCA class method (uncenterend)
K_poly,Kc_poly = create_kernel_matrix(X, polynomial_kernel, degree=2, c=1)
eigenvalues, eigenvectors=eig_sorted(K_poly)
pvalues=np.real(eigenvalues[0:2])
pvectors=np.real(eigenvectors[:,0:2])
D=np.diag(1/np.sqrt(pvalues))
PCA=K_poly@pvectors@D
inter=[slice(0,int(radius[0]*factor)),slice(int(radius[0]*factor),radius[1]*\
    factor),slice(radius[1]*factor,PCA.shape[0])]
for c in range(categories):
    plt.scatter(PCA[inter[c],0],PCA[inter[c],1],s=2,color=color[c])
plt.title("Poly Kernel PCA (Uncentered)")
plt.xlabel('Component 1')
plt.ylabel('Component 2')
plt.axis('equal')
plt.show()
```



[29]:
```
#RBF PCA (centered)
K_rbf,Kc_rbf = create_kernel_matrix(X, rbf_kernel,gamma=.01)
eigenvalues, eigenvectors=eig_sorted(Kc_rbf)
pvalues=np.real(eigenvalues[0:2])
pvectors=np.real(eigenvectors[:,0:2])
D=np.diag(1/np.sqrt(pvalues))
PCA=Kc_rbf@pvectors@D
inter=[slice(0,int(radius[0]*factor)),slice(int(radius[0]*factor),radius[1]*\
    factor),slice(radius[1]*factor,PCA.shape[0])]
for c in range(categories):
    plt.scatter(PCA[inter[c],0],PCA[inter[c],1],s=2,color=color[c])
plt.title("RBF Kernel PCA (Centered)")
plt.xlabel('Component 1')
plt.ylabel('Component 2')
plt.xlim(-1,1)
plt.ylim(-1,1)
plt.axis('equal')
plt.show()
```



[30]:
```
#RBF PCA (uncentered)
K_rbf,Kc_rbf = create_kernel_matrix(X, rbf_kernel,gamma=.01)
eigenvalues, eigenvectors=eig_sorted(K_rbf)
pvalues=eigenvalues[0:2]
pvectors=eigenvectors[:,0:2]
D=np.diag(1/np.sqrt(pvalues))
PCA=K_rbf@pvectors@D
inter=[slice(0,int(radius[0]*factor)),slice(int(radius[0]*factor),radius[1]*\
    factor),slice(radius[1]*factor,PCA.shape[0])]
for c in range(categories):
    plt.scatter(PCA[inter[c],0],PCA[inter[c],1],s=2,color=color[c])
plt.title("RBF Kernel PCA (Uncentered)")
plt.xlabel('Component 1')
plt.ylabel('Component 2')
plt.xlim(-1,1)
plt.ylim(-1,1)
plt.axis('equal')
plt.show()
```


RBF Kernel PCA (Uncentered)
Component 2
Component 1
0.6
0.4
0.2
0.0
−0.2
−0.4
−0.6
0.0
0.2
0.4
0.6
0.8
1.0
1.2
1.4
1.6